# Configuration

## NetApp Solutions

NetApp
March 19, 2024

This PDF was generated from https://docs.netapp.com/zh-cn/netapp-solutions/containers/rh-os-n_use_case_multitenancy_configuration_prerequisites.html on March 19, 2024. Always check docs.netapp.com for the latest.

# 目录

Configuration . . . . . 1
Configuration . . . . . 1
配置： cluster-admin 任务 . . . . . 1
配置： storage-admin 任务 . . . . . 5
验证 . . . . . 8
扩展：添加更多项目 . . . . . 12

# Configuration

对于任何多租户解决方案，任何用户都无法访问比所需更多的集群资源。因此，要在多租户配置中配置的整个资源集将在集群管理员，存储管理员和处理每个项目的开发人员之间进行划分。

下表概括了不同用户要执行的不同任务：

| Role | 任务 |
| --- | --- |
| * 集群管理 * | 为不同的应用程序或工作负载创建项目 |
| | 为 storage-admin 创建 ClusterRoles 和 RoleBindings |
| | 为分配对特定项目的访问权限的开发人员创建角色和角色绑定 |
| | [ 可选 ] 配置项目以在特定节点上计划 Pod |
| * 存储管理 * | 在 NetApp ONTAP 上创建 SVM |
| | 创建 Trident 后端 |
| | 创建 StorageClasses |
| | 创建存储 ResourceQuotas |
| * 开发人员 * | 验证对已分配项目中的 PVC 或 Pod 的创建或修补访问权限 |
| | 验证对在其他项目中创建或修补 PVC 或 Pod 的访问权限 |
| | 验证对查看或编辑项目， ResourceQuotas 和 StorageClasses 的访问权限 |

# Configuration

## 前提条件

- NetApp ONTAP 集群。
- Red Hat OpenShift 集群
- 集群上安装的 Trident 。
- 安装了 tridentctl 和 oc 工具并将其添加到 $path 中的管理工作站。
- 对 ONTAP 的管理员访问权限。
- 对 OpenShift 集群的集群管理员访问。
- 集群已与身份提供程序集成。
- 身份提供程序经过配置，可以有效区分不同团队中的用户。

# 配置： cluster-admin 任务

Red Hat OpenShift cluster-admin 执行以下任务：

1. 以 cluster-admin 身份登录到 Red Hat OpenShift 集群。
2. 创建两个与不同项目对应的项目。

```
oc create namespace project-1
oc create namespace project-2
```

3. 为 project-1 创建开发人员角色。

```
cat << EOF | oc create -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: project-1
  name: developer-project-1
rules:
  - verbs:
      - '*'
    apiGroups:
      - apps
      - batch
      - autoscaling
      - extensions
      - networking.k8s.io
      - policy
      - apps.openshift.io
      - build.openshift.io
      - image.openshift.io
      - ingress.operator.openshift.io
      - route.openshift.io
      - snapshot.storage.k8s.io
      - template.openshift.io
    resources:
      - '*'
  - verbs:
      - '*'
    apiGroups:
      - ''
    resources:
      - bindings
      - configmaps
      - endpoints
      - events
      - persistentvolumeclaims
      - pods
```

```
      - pods/log
      - pods/attach
      - podtemplates
      - replicationcontrollers
      - services
      - limitranges
      - namespaces
      - componentstatuses
      - nodes
  - verbs:
      - '*'
    apiGroups:
      - trident.netapp.io
    resources:
      - tridentsnapshots
EOF
```

本节中提供的角色定义只是一个示例。必须根据最终用户要求定义开发人员角色。

1. 同样，为 project-2 创建开发人员角色。
2. 所有 OpenShift 和 NetApp 存储资源通常由存储管理员管理。存储管理员的访问由安装 Trident 时创建的 Trident 操作员角色控制。此外，存储管理员还需要访问 ResourceQuotas 来控制存储的使用方式。
3. 在集群中的所有项目中创建一个用于管理 ResourceQuotas 的角色，以将其连接到存储管理员：

```
cat << EOF | oc create -f -
kind: ClusterRole
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: resource-quotas-role
rules:
  - verbs:
      - '*'
    apiGroups:
      - ''
    resources:
      - resourcequotas
  - verbs:
      - '*'
    apiGroups:
      - quota.openshift.io
    resources:
      - '*'
EOF
```

4. 确保集群与组织的身份提供程序集成，并且用户组与集群组同步。以下示例显示身份提供程序已与集群集成并与用户组同步。

```
$ oc get groups
NAME                          USERS
ocp-netapp-storage-admins     ocp-netapp-storage-admin
ocp-project-1                 ocp-project-1-user
ocp-project-2                 ocp-project-2-user
```

1. 为存储管理员配置 ClusterRoleBindings 。

```
cat << EOF | oc create -f -
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: netapp-storage-admin-trident-operator
subjects:
  - kind: Group
    apiGroup: rbac.authorization.k8s.io
    name: ocp-netapp-storage-admins
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: trident-operator
---
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: netapp-storage-admin-resource-quotas-cr
subjects:
  - kind: Group
    apiGroup: rbac.authorization.k8s.io
    name: ocp-netapp-storage-admins
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: resource-quotas-role
EOF
```

对于存储管理员，必须绑定两个角色： Trident 操作员和资源配额。

1. 为开发人员创建 RoleBindings ，将开发人员项目 1 角色绑定到项目 1 中的相应组（ OCP-project-1 ）。

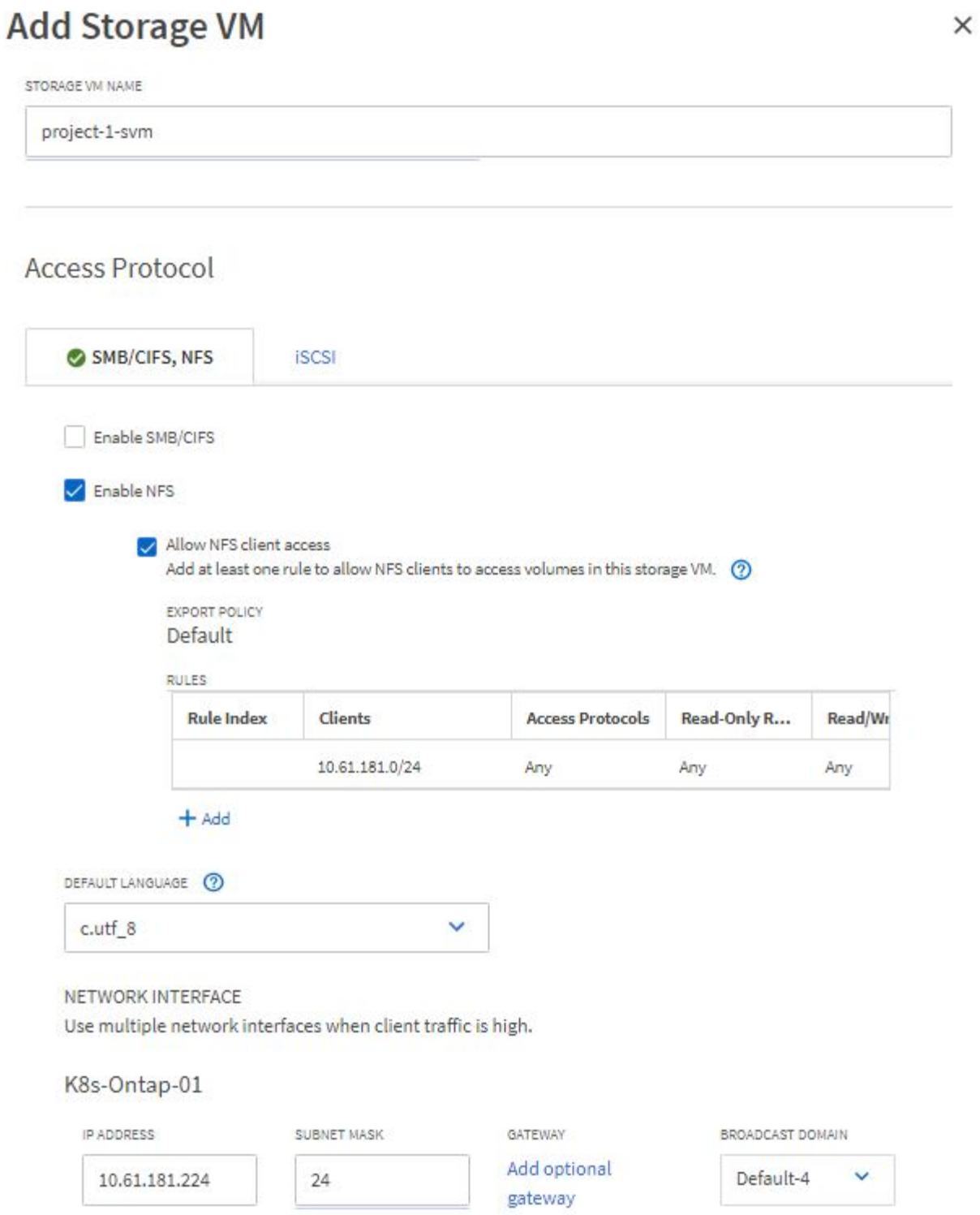
```
cat << EOF | oc create -f -
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: project-1-developer
  namespace: project-1
subjects:
  - kind: Group
    apiGroup: rbac.authorization.k8s.io
    name: ocp-project-1
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: developer-project-1
EOF
```

2. 同样，为开发人员创建 RoleBindings ，将开发人员角色绑定到 project-2 中的相应用户组。

# 配置： storage-admin 任务

存储管理员必须配置以下资源：

1. 以管理员身份登录到 NetApp ONTAP 集群。
2. 导航到存储 > Storage VM ，然后单击添加。通过提供所需的详细信息，创建两个 SVM ，一个用于 project-1 ，另一个用于 project-2 。此外，还可以创建 vsadmin 帐户来管理 SVM 及其资源。




1. 以存储管理员身份登录到 Red Hat OpenShift 集群。
2. 为 project-1 创建后端，并将其映射到专用于该项目的 SVM 。NetApp 建议使用 SVM 的 vsadmin 帐户将后端连接到 SVM ，而不是使用 ONTAP 集群管理员。

```
cat << EOF | tridentctl -n trident create backend -f
{
    "version": 1,
    "storageDriverName": "ontap-nas",
    "backendName": "nfs_project_1",
    "managementLIF": "172.21.224.210",
    "dataLIF": "10.61.181.224",
    "svm": "project-1-svm",
    "username": "vsadmin",
    "password": "NetApp123"
}
EOF
```

在此示例中，我们使用的是 ontap-NAS 驱动程序。根据使用情形创建后端时，请使用相应的驱动程序。

我们假定 Trident 已安装在 Trident 项目中。

1. 同样，为 project-2 创建 Trident 后端，并将其映射到专用于 project-2 的 SVM 。
2. 接下来，创建存储类。为 project-1 创建存储类，并通过设置 storagePools 参数将其配置为使用后端专用于 project-1 的存储池。

```
cat << EOF | oc create -f -
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: project-1-sc
provisioner: csi.trident.netapp.io
parameters:
  backendType: ontap-nas
  storagePools: "nfs_project_1:.*"
EOF
```

3. 同样，为 project-2 创建一个存储类，并将其配置为使用专用于 project-2 的后端存储池。
4. 创建 ResourceQuota 以限制 project-1 中的资源，从而从专用于其他项目的存储库请求存储。

```
cat << EOF | oc create -f -
kind: ResourceQuota
apiVersion: v1
metadata:
  name: project-1-sc-rq
  namespace: project-1
spec:
  hard:
    project-2-sc.storageclass.storage.k8s.io/persistentvolumeclaims: 0
EOF
```

5. 同样，也可以创建 ResourceQuota 来限制项目 2 中的资源，以便从专用于其他项目的存储库请求存储。

# 验证

要验证在上述步骤中配置的多租户架构，请完成以下步骤：

## 验证在分配的项目中创建 **PVC** 或 **Pod** 的访问权限

1. 以项目 1 中的 OCP-project-1-user 和开发人员身份登录。
2. 检查访问权限以创建新项目。

```
oc create ns sub-project-1
```

3. 使用分配给 project-1 的 storageclass 在 project-1 中创建 PVC 。

```
cat << EOF | oc create -f -
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: test-pvc-project-1
  namespace: project-1
  annotations:
    trident.netapp.io/reclaimPolicy: Retain
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: project-1-sc
EOF
```

4. 检查与 PVC 关联的 PV 。

```
oc get pv
```

5. 验证 PV 及其卷是否已在 NetApp ONTAP 上专用于 project-1 的 SVM 中创建。

```
volume show -vserver project-1-svm
```

6. 在 project-1 中创建 POD ，然后挂载上一步创建的 PVC 。

```
cat << EOF | oc create -f -
kind: Pod
apiVersion: v1
metadata:
  name: test-pvc-pod
  namespace: project-1
spec:
  volumes:
    - name: test-pvc-project-1
      persistentVolumeClaim:
       claimName: test-pvc-project-1
  containers:
    - name: test-container
      image: nginx
      ports:
        - containerPort: 80
          name: "http-server"
      volumeMounts:
        - mountPath: "/usr/share/nginx/html"
          name: test-pvc-project-1
EOF
```

7. 检查 POD 是否正在运行以及是否已挂载卷。

```
oc describe pods test-pvc-pod -n project-1
```

## 验证在其他项目中创建 **PVC** 或 **Pod** 的访问权限，或者使用专用于另一项目的资源

1. 以项目 1 中的 OCP-project-1-user 和开发人员身份登录。
2. 使用分配给 project-2 的 storageclass 在 project-1 中创建 PVC 。

```
cat << EOF | oc create -f -
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: test-pvc-project-1-sc-2
  namespace: project-1
  annotations:
    trident.netapp.io/reclaimPolicy: Retain
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: project-2-sc
EOF
```

3. 在 project-2 中创建 PVC 。

```
cat << EOF | oc create -f -
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: test-pvc-project-2-sc-1
  namespace: project-2
  annotations:
    trident.netapp.io/reclaimPolicy: Retain
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: project-1-sc
EOF
```

4. 请确保未创建 PVC `test-vpa-project-1-sc-2` 和 `test-vpa-project-2-sc-1` 。

```
oc get pvc -n project-1
oc get pvc -n project-2
```

5. 在 project-2 中创建 POD 。

```
cat << EOF | oc create -f -
kind: Pod
apiVersion: v1
metadata:
  name: test-pvc-pod
  namespace: project-1
spec:
  containers:
    - name: test-container
      image: nginx
      ports:
        - containerPort: 80
          name: "http-server"
EOF
```

### 验证对查看和编辑项目， **ResourceQuotas** 和 **StorageClasses** 的访问权限

1. 以项目 1 中的 OCP-project-1-user 和开发人员身份登录。
2. 检查访问权限以创建新项目。

```
oc create ns sub-project-1
```

3. 验证对查看项目的访问权限。

```
oc get ns
```

4. 检查用户是否可以在 project-1 中查看或编辑 ResourceQuotas 。

```
oc get resourcequotas -n project-1
oc edit resourcequotas project-1-sc-rq -n project-1
```

5. 验证用户是否有权查看存储器。

```
oc get sc
```

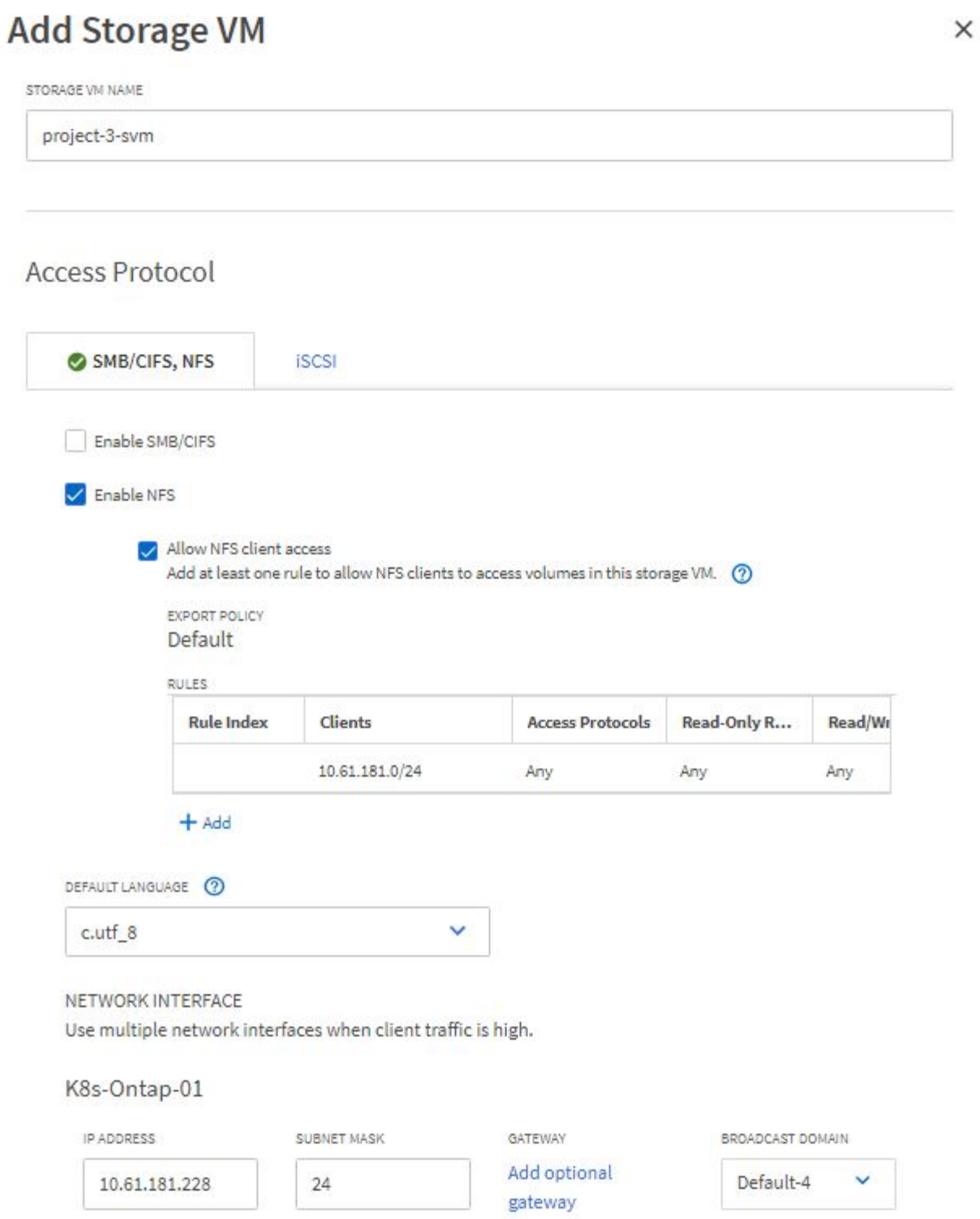
6. 检查访问权限以描述存储器。
7. 验证用户的访问权限以编辑存储器库。

```
oc edit sc project-1-sc
```

# 扩展：添加更多项目

在多租户配置中，使用存储资源添加新项目需要进行额外配置，以确保不会违反多租户要求。要在多租户集群中添加更多项目，请完成以下步骤：

1. 以存储管理员身份登录到 NetApp ONTAP 集群。
2. 导航到 `Storage → Storage VM` ，然后单击 `Add` 。创建一个专用于 project-3 的新 SVM 。此外，还可以创建 vsadmin 帐户来管理 SVM 及其资源。




1. 以集群管理员身份登录到 Red Hat OpenShift 集群。
2. 创建新项目。

```
oc create ns project-3
```

3. 确保已在 IdP 上为 project-3 创建用户组并与 OpenShift 集群同步。

```
oc get groups
```

4. 为 project-3 创建开发人员角色。

```
cat << EOF | oc create -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: project-3
  name: developer-project-3
rules:
  - verbs:
      - '*'
    apiGroups:
      - apps
      - batch
      - autoscaling
      - extensions
      - networking.k8s.io
      - policy
      - apps.openshift.io
      - build.openshift.io
      - image.openshift.io
      - ingress.operator.openshift.io
      - route.openshift.io
      - snapshot.storage.k8s.io
      - template.openshift.io
    resources:
      - '*'
  - verbs:
      - '*'
    apiGroups:
      - ''
    resources:
      - bindings
      - configmaps
      - endpoints
      - events
      - persistentvolumeclaims
      - pods
      - pods/log
      - pods/attach
      - podtemplates
```

```
      - replicationcontrollers
      - services
      - limitranges
      - namespaces
      - componentstatuses
      - nodes
  - verbs:
      - '*'
    apiGroups:
      - trident.netapp.io
    resources:
      - tridentsnapshots
EOF
```

本节中提供的角色定义只是一个示例。必须根据最终用户要求定义开发人员角色。

1. 在 project-3 中为开发人员创建 RoleBinding. 将开发人员项目 3 角色绑定到 project-3 中的相应组（ OCP-project-3 ）。

```
cat << EOF | oc create -f -
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: project-3-developer
  namespace: project-3
subjects:
  - kind: Group
    apiGroup: rbac.authorization.k8s.io
    name: ocp-project-3
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: developer-project-3
EOF
```

2. 以存储管理员身份登录到 Red Hat OpenShift 集群
3. 创建 Trident 后端并将其映射到专用于 project-3 的 SVM 。NetApp 建议使用 SVM 的 vsadmin 帐户将后端连接到 SVM ，而不是使用 ONTAP 集群管理员。

```
cat << EOF | tridentctl -n trident create backend -f
{
    "version": 1,
    "storageDriverName": "ontap-nas",
    "backendName": "nfs_project_3",
    "managementLIF": "172.21.224.210",
    "dataLIF": "10.61.181.228",
    "svm": "project-3-svm",
    "username": "vsadmin",
    "password": "NetApp!23"
}
EOF
```

在此示例中，我们使用的是 ontap-NAS 驱动程序。使用相应的驱动程序根据使用情形创建后端。

我们假定 Trident 已安装在 Trident 项目中。

1. 为 project-3 创建存储类，并将其配置为使用专用于 project-3 的后端存储池。

```
cat << EOF | oc create -f -
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: project-3-sc
provisioner: csi.trident.netapp.io
parameters:
  backendType: ontap-nas
  storagePools: "nfs_project_3:.*"
EOF
```

2. 创建 ResourceQuota 以限制项目 3 中的资源，从而从专用于其他项目的存储库请求存储。

```
cat << EOF | oc create -f -
kind: ResourceQuota
apiVersion: v1
metadata:
  name: project-3-sc-rq
  namespace: project-3
spec:
  hard:
    project-1-sc.storageclass.storage.k8s.io/persistentvolumeclaims: 0
    project-2-sc.storageclass.storage.k8s.io/persistentvolumeclaims: 0
EOF
```

3. 在其他项目中修补 ResourceQuotas ，以限制这些项目中的资源从专用于项目 3 的存储库访问存储。

```
oc patch resourcequotas project-1-sc-rq -n project-1 --patch
'{"spec":{"hard":{ "project-3-
sc.storageclass.storage.k8s.io/persistentvolumeclaims": 0}}}'
oc patch resourcequotas project-2-sc-rq -n project-2 --patch
'{"spec":{"hard":{ "project-3-
sc.storageclass.storage.k8s.io/persistentvolumeclaims": 0}}}'
```

版权信息

版权所有 © 2024 NetApp, Inc.。保留所有权利。中国印刷。未经版权所有者事先书面许可，本文档中受版权保护的任何部分不得以任何形式或通过任何手段（图片、电子或机械方式，包括影印、录音、录像或存储在电子检索系统中）进行复制。

从受版权保护的 NetApp 资料派生的软件受以下许可和免责声明的约束：

本软件由 NetApp 按“原样”提供，不含任何明示或暗示担保，包括但不限于适销性以及针对特定用途的适用性的隐含担保，特此声明不承担任何责任。在任何情况下，对于因使用本软件而以任何方式造成的任何直接性、间接性、偶然性、特殊性、惩罚性或后果性损失（包括但不限于购买替代商品或服务；使用、数据或利润方面的损失；或者业务中断），无论原因如何以及基于何种责任理论，无论出于合同、严格责任或侵权行为（包括疏忽或其他行为），NetApp 均不承担责任，即使已被告知存在上述损失的可能性。

NetApp 保留在不另行通知的情况下随时对本文档所述的任何产品进行更改的权利。除非 NetApp 以书面形式明确同意，否则 NetApp 不承担因使用本文档所述产品而产生的任何责任或义务。使用或购买本产品不表示获得 NetApp 的任何专利权、商标权或任何其他知识产权许可。

本手册中描述的产品可能受一项或多项美国专利、外国专利或正在申请的专利的保护。

有限权利说明：政府使用、复制或公开本文档受 DFARS 252.227-7013（2014 年 2 月）和 FAR 52.227-19（2007 年 12 月）中“技术数据权利 — 非商用”条款第 (b)(3) 条规定的限制条件的约束。

本文档中所含数据与商业产品和/或商业服务（定义见 FAR 2.101）相关，属于 NetApp, Inc. 的专有信息。根据本协议提供的所有 NetApp 技术数据和计算机软件具有商业性质，并完全由私人出资开发。 美国政府对这些数据的使用权具有非排他性、全球性、受限且不可撤销的许可，该许可既不可转让，也不可再许可，但仅限在与交付数据所依据的美国政府合同有关且受合同支持的情况下使用。除本文档规定的情形外，未经 NetApp, Inc. 事先书面批准，不得使用、披露、复制、修改、操作或显示这些数据。美国政府对国防部的授权仅限于 DFARS 的第 252.227-7015(b)（2014 年 2 月）条款中明确的权利。

商标信息

NetApp、NetApp 标识和 http://www.netapp.com/TM 上所列的商标是 NetApp, Inc. 的商标。其他公司和产品名称可能是其各自所有者的商标。